

JavaScript Core for Playwright

Beginner Curriculum • Phase 1 • ~8–10 Hours

A practical, tester-focused JavaScript learning path for someone coming from Java. The goal is not to become a JavaScript developer in one week. The goal is to become comfortable enough with JavaScript to read, write, debug, and maintain Playwright tests.

Senior-programmer approach: Learn the 20% of JavaScript that appears in 80% of test automation code. Write code after every concept. Do not memorize syntax without using it.

Phase	Topic	Suggested time
1	Variables, types, operators	1 hr
2	Functions + arrow functions	1.25 hr
3	Objects + destructuring	1.25 hr
4	Arrays + array methods	1.5 hr
5	Template literals	30 min
6	Conditions + loops	1 hr
7	Playwright-oriented practice	1.5–2 hr

1. How to use this curriculum

Because you already know Java, you should not learn JavaScript like a complete programming beginner. Your Java knowledge gives you the concepts; your job is to learn JavaScript's syntax, behavior, and idioms.

- **Rule 1 — Type every example:** don't just read the code.
- **Rule 2 — Predict first:** before running code, say what you expect it to print.
- **Rule 3 — Use Playwright context:** connect every concept to a test problem.
- **Rule 4 — Prefer const:** use const by default; use let only when reassignment is required.
- **Rule 5 — Don't overlearn:** classes, prototypes, generators, advanced functional programming, and metaprogramming can wait.

Java-to-JavaScript mental translation

Java	JavaScript	What to remember
String name = "Sibi";	const name = "Sibi";	No declared type before the variable.
int age = 28;	const age = 28;	number covers integers and decimals.
boolean active = true;	const active = true;	boolean is similar.
null	null	Intentional absence of value.
Method	function / arrow function	Functions are first-class values.
List<String>	const names = ["A", "B"]	Array is the common list structure.
Map<String,Object>	const user = { name: "A" }	Object is often used for structured data.

2. Variables & Types — let, const, and basic values

JavaScript variables do not require you to declare a type. In modern JavaScript, start with **const**. If the variable must change, use **let**. Avoid **var** in modern test code.

Basic variables

```
const userName = "Sibi";
let retryCount = 0;
const isLoggedIn = true;
const score = 98.5;
const missingValue = null;
let notAssigned;

retryCount = retryCount + 1;

console.log(userName);
console.log(typeof score);           // "number"
console.log(typeof isLoggedIn);      // "boolean"
console.log(typeof notAssigned);     // "undefined"
```

Important behavior

- **const** prevents reassignment of the variable binding. It does not make an object immutable.
- **let** allows reassignment.
- **undefined** usually means a value has not been assigned or a property does not exist.
- **null** is an intentional 'no value' value.
- JavaScript has one **number** type for ordinary integers and decimals.

const object can still be modified

```
const user = { name: "Sibi", role: "Tester" };

user.role = "Automation Engineer"; // allowed
// user = {};                      // not allowed
```

Tester connection

Playwright code frequently stores pages, locators, test data, URLs, credentials, and configuration values in variables. Using `const` consistently makes tests easier to reason about.

Practice: Create variables for a URL, browser name, test user, login status, and retry count. Print their values and types.

3. Operators, equality, and truthy/falsy

Use `===` and `!==` for equality. They compare both value and type, which avoids many surprising conversions.

Equality

```
console.log(5 === 5);      // true
console.log(5 === "5");    // false

console.log(5 !== "5");    // true
console.log("PASS" === "PASS"); // true
```

Truthy and falsy

In conditions, JavaScript converts a value to true or false. Common falsy values are: **false**, **0**, **""**, **null**, **undefined**, **NaN**. Most other values are truthy, including empty arrays and empty objects.

Truthy/falsy in a test

```
const errorMessage = "";

if (errorMessage) {
  console.log("Error exists");
} else {
  console.log("No error");
}

const response = null;

if (!response) {
  console.log("Response is missing");
}
```

Practice: Write a condition that prints 'Login successful' only when `isLoggedIn` is true. Then test it with false.

4. Functions — the most important skill for automation

A function groups reusable behavior. In test automation, functions reduce duplicated actions such as login, search, navigation, API setup, and data creation.

Normal function

```
function add(a, b) {  
    return a + b;  
}  
  
const result = add(10, 20);  
console.log(result); // 30
```

Arrow function

```
const add = (a, b) => {  
    return a + b;  
};  
  
const multiply = (a, b) => a * b;
```

How to read an arrow function

- **(a, b)** → parameters
- **=>** → arrow-function syntax
- **{ ... }** → function body
- If there is one expression, braces and return can often be omitted.

Why Playwright uses arrows everywhere

```
test("login works", async ({ page }) => {  
    await page.goto("https://example.com");  
    await page.getByLabel("Username").fill("tester");  
    await page.getByLabel("Password").fill("secret");  
});
```

The **async** keyword and **await** are important for Playwright. You do not need to master asynchronous JavaScript today, but you must recognize the syntax. Playwright operations such as `goto()`, `click()`, `fill()`, and `expect()` commonly participate in asynchronous flows.

Practice: Create a function `loginMessage(username)` that returns `'Welcome, '`. Then rewrite it as an arrow function.

Expected solution

```
function loginMessage(username) {  
    return `Welcome, ${username}`;  
}  
  
const loginMessage2 = (username) => `Welcome, ${username}`;
```

5. Objects & destructuring

Objects are JavaScript's everyday way to represent structured data. Think of them as a lightweight record containing properties and values.

Object literal

```
const user = {  
    name: "Sibi",  
    role: "Automation Tester",  
    experience: 5,  
    active: true  
};  
  
console.log(user.name);  
console.log(user["role"]);
```

Dot vs bracket access

- **user.name** is the normal readable form.
- **user["role"]** is useful when the property name is dynamic.
- Example: `user[fieldName]` where `fieldName` comes from a variable.

Destructuring

```
const user = {
  name: "Sibi",
  role: "Automation Tester"
};

const { name, role } = user;

console.log(name);
console.log(role);
```

Playwright connection

Playwright test fixtures often expose objects. You will commonly see patterns such as **const { page } =** This is destructuring: extracting a property from an object into a variable.

Fixture-style destructuring

```
async function openHome({ page }) {
  await page.goto("https://example.com");
}
```

Practice: Create a `testUser` object with `username`, `password`, `role`, and `expectedTitle`. Destructure `username` and `expectedTitle` into separate variables.

6. Arrays & array methods

Arrays hold ordered collections. For test automation, you will use arrays for test data, lists of elements, menu items, API responses, and sets of expected values.

Basic array

```
const browsers = ["chromium", "firefox", "webkit"];

console.log(browsers[0]); // chromium
console.log(browsers.length); // 3
```

Core methods you should master

Method	Purpose	Typical tester use
<code>map()</code>	Transform every item	Convert test data into another shape
<code>filter()</code>	Keep matching items	Find active users / matching rows
<code>forEach()</code>	Run code for each item	Perform repeated checks
<code>find()</code>	Return first match	Find one matching test record
<code>includes()</code>	Check membership	Verify expected status/value

`map()`

```
const users = [
  { name: "Asha", active: true },
  { name: "Ravi", active: false },
  { name: "John", active: true }
];

const names = users.map(user => user.name);

console.log(names);
// ["Asha", "Ravi", "John"]
```

`filter()`

```

const activeUsers = users.filter(user => user.active);

console.log(activeUsers);
// Asha and John

find() and includes()
const ravi = users.find(user => user.name === "Ravi");

const statuses = ["PASS", "FAIL", "SKIPPED"];

console.log(ravi);
console.log(statuses.includes("PASS")); // true

forEach()
users.forEach(user => {
  console.log(`Checking ${user.name}`);
});

```

The key difference

map creates a new array. **filter** creates a smaller array. **find** returns one item (or undefined). **forEach** executes code but is not used to create a transformed array.

Practice: Given an array of test cases, use `filter()` to get only P0 tests, `map()` to extract their names, and `includes()` to check whether 'Login' exists.

7. Template literals — dynamic strings

Template literals use backticks. They are especially useful for dynamic URLs, messages, selectors, logs, and test names.

Template literal

```

const username = "Sibi";
const testName = `Login test for ${username}`;

console.log(testName);
// Login test for Sibi

```

Dynamic URL

```

const userId = 101;
const url = `https://example.com/users/${userId}`;

console.log(url);

```

Playwright connection

Dynamic test data

```

const product = "Laptop";
await page.getByRole("link", { name: `Buy ${product}` }).click();

```

Practice: Build a message using name, environment, and browser variables: 'Running in on '.

8. Conditions & loops

if / else

```

const status = "PASS";

if (status === "PASS") {
  console.log("Test passed");
} else {
  console.log("Test failed");
}

```

Multiple conditions

```

const statusCode = 200;

if (statusCode === 200) {
  console.log("Success");
}

```

```
} else if (statusCode === 401) {
  console.log("Unauthorized");
} else {
  console.log("Unexpected response");
}
```

for...of

Use **for...of** when you want to iterate through array values. It is usually clearer than an index-based loop for test code.

Loop through test data

```
const users = ["admin", "manager", "tester"];

for (const user of users) {
  console.log(`Testing login for ${user}`);
}
```

Playwright-style iteration

```
const menuItems = ["Home", "Products", "Reports"];

for (const item of menuItems) {
  await page.getByRole("link", { name: item }).click();
}
```

Practice: Create an array of three page titles. Loop through it and print 'Validating: '. Add an if condition that prints PASS only for a chosen title.

9. Integration exercise — think like an automation engineer

Now combine variables, objects, arrays, functions, destructuring, template literals, conditions, and loops into one small piece of automation-oriented JavaScript.

Mini test-data model

```
const testCases = [
  {
    name: "Valid login",
    username: "admin",
    expectedStatus: "PASS"
  },
  {
    name: "Invalid login",
    username: "wrong-user",
    expectedStatus: "FAIL"
  },
  {
    name: "Locked user",
    username: "locked-user",
    expectedStatus: "FAIL"
  }
];

const validCases = testCases.filter(
  testCase => testCase.expectedStatus === "PASS"
);

for (const testCase of validCases) {
  const { name, username } = testCase;

  console.log(`Running: ${name}`);
  console.log(`Username: ${username}`);
}
```

What happened here?

- **const** stores the testCases array.
- Objects represent individual test cases.
- **filter()** selects only PASS cases.
- **for...of** iterates over the selected cases.
- Destructuring extracts name and username.
- Template literals create readable log messages.
- Arrow functions are used inside filter().

10. Playwright-focused examples

Once the JavaScript basics are comfortable, immediately move them into Playwright. The objective is to understand the JavaScript around Playwright APIs—not to memorize every API.

Example 1 — basic test

```
import { test, expect } from "@playwright/test";

test("login page is displayed", async ({ page }) => {
  await page.goto("https://example.com/login");

  const title = await page.title();

  expect(title).toContain("Login");
});
```

Example 2 — object for test data

```
const loginData = {
```

```

    username: "testuser",
    password: "secret123"
  };

  test("user can login", async ({ page }) => {
    await page.goto("https://example.com/login");

    const { username, password } = loginData;

    await page.getByLabel("Username").fill(username);
    await page.getByLabel("Password").fill(password);
    await page.getByRole("button", { name: "Login" }).click();
  });

```

Example 3 — array + loop

```

const menuItems = ["Home", "Products", "Reports"];

test("main menu links are accessible", async ({ page }) => {
  await page.goto("https://example.com");

  for (const item of menuItems) {
    await expect(
      page.getByRole("link", { name: item })
    ).toBeVisible();
  }
});

```

Example 4 — function + template literal

```

const buildUserMessage = (name, role) =>
  `Logged in as ${name} (${role})`;

test("user message", async ({ page }) => {
  const message = buildUserMessage("Sibi", "Tester");
  console.log(message);
});

```

Senior-level habit

Do not create complicated one-line expressions just because JavaScript allows them. Test code is read by the whole QA team. Prefer clear variable names, small functions, and straightforward array methods.

11. Your 8–10 hour study plan

Time	Study	Deliverable
0:00–1:00	Variables, types, operators	10 small console examples
1:00–2:15	Functions + arrow functions	5 reusable functions
2:15–3:30	Objects + destructuring	Test-user object + extraction
3:30–5:00	Arrays + map/filter/find/forEach/includes	10 array exercises
5:00–5:30	Template literals	Dynamic URL + message examples
5:30–6:30	if/else + for...of	Status + test-data loop
6:30–8:00	Playwright integration	3 small Playwright tests
8:00–10:00	Mini project + debugging	Data-driven login test

12. Mini project — Data-driven login validation

Build a small JavaScript data set and use it from a Playwright test. You should be able to explain every line before moving on.

Step 1 — test data

```
const loginUsers = [
  {
    username: "validUser",
    password: "correctPassword",
    expected: "success"
  },
  {
    username: "invalidUser",
    password: "wrongPassword",
    expected: "error"
  }
];
```

Step 2 — helper function

```
const getUserByExpectedResult = (expected) =>
  loginUsers.find(user => user.expected === expected);
```

Step 3 — Playwright test

```
import { test, expect } from "@playwright/test";

test("valid login", async ({ page }) => {
  const user = getUserByExpectedResult("success");

  await page.goto("https://example.com/login");
  await page.getByLabel("Username").fill(user.username);
  await page.getByLabel("Password").fill(user.password);
  await page.getByRole("button", { name: "Login" }).click();

  await expect(page.getByText("Dashboard")).toBeVisible();
});
```

Practice: Extend the project with a second test for invalid login. Add a third user, filter the data by expected result, and print a useful message for each user.

13. Common beginner mistakes to avoid

- **Using var everywhere:** don't. Prefer const/let.
- **Using == instead of ===:** use strict equality unless you have a specific reason not to.
- **Overusing indexes:** prefer for...of or array methods when working with collections.
- **Huge functions:** split repeated behavior into small functions.
- **Ignoring return values:** understand what map(), filter(), find(), and includes() return.
- **Confusing object and array:** [] is an array; {} is an object.
- **Trying to learn everything:** for Playwright, focus on practical JavaScript first.
- **Copy/paste without prediction:** predict output before executing code.

14. Phase 1 completion checklist

- I can explain the difference between const and let.
- I can identify string, number, boolean, null, and undefined.
- I use === and !== confidently.
- I understand common truthy/falsy values.
- I can write normal and arrow functions.
- I understand async/await at a recognition level.
- I can create and access objects.
- I can destructure object properties.
- I can create and manipulate arrays.

- I can use map, filter, forEach, find, and includes.
- I can create dynamic strings with template literals.
- I can write if/else conditions.
- I can iterate arrays with for...of.
- I can read the JavaScript surrounding a Playwright test.
- I can write a small Playwright test without copying every line.

15. What to learn next

After Phase 1, do not immediately jump into advanced JavaScript. Move in this order:

- **Phase 2:** async/await deeply, promises, error handling, modules/import-export, npm basics.
- **Phase 3:** Playwright locators, assertions, fixtures, hooks, page objects, test data, configuration.
- **Phase 4:** advanced Playwright — API testing, authentication state, parallelism, retries, traces, reports.
- **Phase 5:** framework design — reusable utilities, environment configuration, CI/CD, Git/Jenkins/GitHub Actions.
- **Phase 6:** AI-assisted testing — test generation, locator analysis, failure summarization, flaky-test analysis.

Target outcome: You do not need to say 'I know JavaScript' after Phase 1. You should be able to look at a Playwright test and understand why every variable, function, object, array, condition, and loop is there.